Halo 2 Guide

The Halo 2 proof system, from arithmetisation to recursion

m@rek.onl

Abstract

This is the third volume of a four-part series — *Math Guide*, *Crypto Guide*, *Halo 2 Guide*, *Orchard Guide*. Assuming the mathematics of the *Math Guide* and the cryptographic primitives of the *Crypto Guide*, it develops the Halo 2 proof system: the polynomial-IOP design pattern, a rigorous account of knowledge soundness in the algebraic group model, proof-carrying data and accumulation schemes, and finally Halo 2 itself — PLONKish arithmetisation, the permutation and lookup arguments, the inner-product polynomial commitment and the Halo accumulation trick, and the complete protocol.

Contents

1 Introduction

This is the *Halo 2 Guide*, the third volume of a four-part development of the cryptography behind Zcash's Orchard protocol. The first two volumes — the *Math Guide* and the *Crypto Guide* — constructed, from the ground up, the mathematics (finite fields, elliptic curves, polynomials and evaluation domains, probability) and the cryptographic primitives (hash functions, commitments, Σ-protocols and the Fiat–Shamir transform, zero knowledge, polynomial commitment schemes) that this volume assumes. The present volume assembles those ingredients into a *proof system*.

The development proceeds in four steps. First, we make the polynomial-IOP design pattern precise: the recipe "encode a computation as polynomial identities over an evaluation domain, then compile to a succinct argument with a polynomial commitment." Next, we treat knowledge soundness rigorously, including the algebraic group model and the tree-of-transcripts extractor that the inner-product argument requires. We then develop proof-carrying data, incrementally-verifiable computation, and the accumulation schemes that make recursion practical. Finally, we assemble Halo 2 itself: PLONKish arithmetisation, the permutation and lookup arguments, the inner-product polynomial commitment with a worked example and its accumulation (the "Halo trick"), and the complete seven-phase protocol. The *Orchard Guide* subsequently uses this proof system to construct a shielded payment protocol.

2 Polynomial IOPs and the SNARK design pattern

The modern SNARK design pattern factors the protocol into two clean layers: a *polynomial interactive oracle proof* (PIOP) for the relation, and a polynomial commitment scheme (the *Crypto Guide*) that realises the PIOP's oracles. This factoring, due in its current form to Bünz, Fisch, Szepieniec, and others, has substantially clarified the nature of SNARKs.

2.1 The polynomial IOP, formally

Definition 2.1 (Polynomial IOP)**.** A *polynomial interactive oracle proof* for a relation $\mathcal{R}$ is a public-coin interactive protocol where in each round:

- The prover sends one or more *polynomial oracles* $p_i \in \mathbb{F}[X]$ (each of bounded degree).
- The verifier sends a random challenge $c \in \mathbb{F}$.

At the end of the protocol, the verifier may query each oracle at a small number of points, learning the polynomial's evaluations at those points. The verifier's final accept/reject is a deterministic function of the challenges, evaluations, and the public input x.

We define completeness, soundness, and knowledge soundness as usual.

A PIOP differs from a vanilla interactive argument in one essential respect: the verifier does not read the prover's messages in full — it queries only a few evaluations of each polynomial oracle. This is what renders the verifier "succinct" even when the polynomials themselves have large degree.

2.2 Compiling a PIOP into a SNARK

Construction 2.2 (PIOP-to-SNARK compiler)**.** Given a PIOP Π for $\mathcal{R}$, a PCS scheme $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Verify})$ for $\mathbb{F}[X]$, and a random oracle H, the compiler produces the SNARK:

1. Setup: run the PCS setup to obtain pp.
2. Prover: simulate Π, replacing each polynomial oracle p_i by its PCS commitment C_i, and each verifier-challenge / oracle-query round by the corresponding PCS opening. Use Fiat–Shamir (the *Crypto Guide*) to derive challenges from the transcript.

3. Verifier: parse the transcript, recompute Fiat–Shamir challenges, verify each PCS opening, and run the PIOP's final accept/reject check.

Theorem 2.3 (Soundness preservation)**.** *If Π has knowledge soundness κ_Π and the PCS has knowledge binding κ_{PCS}, the compiled SNARK has knowledge soundness $O(\kappa_\Pi + Q \cdot \kappa_{\mathrm{PCS}})$ in the random oracle model, where Q is the adversary's RO-query budget.*

This factoring is of considerable utility: research on PIOPs (PLONK, Marlin, Spartan, HyperPlonk, ...) proceeds independently of research on polynomial commitment schemes, and one may combine any compatible pair to produce a new SNARK with mixed-and-matched properties. This document develops the one PCS that Halo 2 uses — the inner-product argument (§5.4) — and treats the PIOP/PCS split only insofar as it clarifies the Halo 2 construction.

2.3 A worked example: a PIOP for univariate evaluation

To render the abstraction concrete, consider a minimal PIOP for the simplest non-trivial relation: "the prover knows a polynomial p of degree $\leq d$ such that $p(z) = y$ for a public (z, y)."

Construction 2.4 (PIOP for univariate evaluation)**.** On input (z, y) and prover witness p:

1. Prover sends the polynomial oracle p and the quotient oracle $q := (p - y)/(X - z)$.
2. Verifier samples a random challenge $r \in \mathbb{F}$.
3. Verifier queries p and q at r.
4. Verifier accepts iff $p(r) - y = q(r) \cdot (r - z)$.

Completeness is immediate (q is a polynomial since $p(z) - y = 0$). Soundness follows from Schwartz–Zippel: if $(X - z) \nmid (p(X) - y)$, then $p(X) - y - q(X)(X - z)$ is a non-zero polynomial of degree $\leq d$, and its random evaluation is zero with probability $\leq d/|\mathbb{F}|$.

Compiling this PIOP with the IPA-PCS, applying Fiat–Shamir, yields a non-interactive SNARK for the same relation. The PCS handles polynomial hiding and binding; the PIOP handles the relation logic. This separation is precisely the modular SNARK design pattern.

2.4 PLONK as a PIOP

PLONK (§5.1) is a more elaborate PIOP. Its oracles are:

- One advice polynomial per advice column.
- A permutation running-product polynomial Z_{perm}.
- One lookup running-product polynomial per lookup.
- A quotient polynomial h (split into chunks).

Its challenges are β, γ (for the permutation argument), θ (for lookup column compression), y (for the gate-equation combination), x (the final evaluation point). Its final check is the PLONK gate equation $C(x) = h(x) \cdot Z_H(x)$, which the verifier checks from the evaluations of the oracles at x (and possibly ωx).

This is precisely the PIOP that phases 1–6 of the Halo 2 protocol of §5.6 compile; phase 7 is the IPA-PCS opening of the combined polynomial.

3 A rigorous treatment of knowledge soundness

We now develop knowledge soundness with the care needed for arguments about modern SNARKs. The treatment combines the rewinding-extractor framework (the *Crypto Guide*) with the algebraic group model.

3.1 Definitional refinements

The bare definition of the *Crypto Guide* suffices for Σ-protocols but is awkward for compound systems. We adopt the following standard refinement.

Definition 3.1 (Witness-extended emulation)**.** An interactive argument $(\mathcal{P}, \mathcal{V})$ has *witness-extended emulation* if there is a PPT emulator $\mathcal{E}$ such that for every PPT prover $\mathcal{P}^*$ and every input x:

- Run $\mathcal{P}^*$ honestly with $\mathcal{V}$; let τ be the transcript and b the verifier's accept/reject bit.
- Run $\mathcal{E}^{\mathcal{P}^*}(x)$; it outputs (τ', b', w).

The distributions of (τ, b) and (τ', b') are computationally indistinguishable, and moreover whenever $b' = 1$, $(x, w) \in \mathcal{R}$ except with negligible probability.

WEE captures simultaneously two properties: "the extractor produces a witness whenever the protocol accepts" and "the extractor's external behaviour resembles a normal protocol execution." It is the appropriate notion for arguments embedded in larger protocols (e.g., for batched proofs or for verifying Halo 2 proofs within a Halo 2 verifier circuit).

3.2 The algebraic group model

The IPA's knowledge-soundness proof proceeds through the tree extractor of §5.4, with knowledge error $O(d/|\mathbb{F}|)$. The number of prover invocations the extractor makes is $O(d)$, but converting those invocations into a standard-model reduction to DL via iterated forking is loose: each of the $\log d$ rounds contributes a multiplicative blow-up, so the standard-model extractor runs in quasi-polynomial time $d^{O(\log d)}$ rather than strict polynomial time. The algebraic group model removes this loss.

The *algebraic group model* (AGM; Fuchsbauer, Kiltz, Loss; CRYPTO 2018) provides a tighter analysis by restricting the adversary's group operations.

Definition 3.2 (Algebraic adversary)**.** An *algebraic adversary* against a group $\mathbb{G}$ is a PPT algorithm that, whenever it outputs a group element $P \in \mathbb{G}$, additionally outputs an explicit linear combination $P = \sum_i [a_i] Q_i$ expressing P as a combination of previously-seen group elements Q_i.

The AGM is intermediate between the standard model (no restrictions on the adversary) and the generic group model (the adversary can access group operations only via an oracle). Many real adversaries are algebraic — they compute group elements via known scalar multiplications and additions — so AGM analyses are often sharper than standard-model analyses while remaining heuristically conservative.

For Halo 2, BCMS20 conduct the accumulation analysis in the AGM. The resulting extractor is polynomial in the round count $k = \log d$, neither exponential nor quasi-polynomial: each algebraic adversary's group-element outputs come with explicit linear combinations, so the extractor reads off the witness from the combinations.

3.3 Tree-extractor analysis

To render the tree extractor concrete, consider the key step at each round of the IPA halving (see §5.4):

Lemma 3.3 (Single-round extraction)**.** *In a single round of the IPA at depth j, given three accepting sub-transcripts that share the round's first message (L_j, R_j) but have three distinct challenges u_j, the extractor recovers the round-j witness halves $\vec{a}_{\mathrm{lo}}^{(j)}, \vec{a}_{\mathrm{hi}}^{(j)}$ together with the consistency of the L_j, R_j cross-terms via 3×3 Vandermonde inversion (rows $(u_j^{-2}, 1, u_j^2)$), since the folded relation is quadratic in u_j.*

Sketch. The three accepting transcripts give three accepting verifier equations. Because the folded relation is quadratic in u_j (the three monomials $u_j^{-2}, 1, u_j^2$), treating these as three linear equations in three unknowns (the cross-terms and the folded commitment), the system has a unique solution because the 3×3 Vandermonde determinant with rows $(u_j^{-2}, 1, u_j^2)$ is non-zero when the three challenges are distinct (and $u_j \neq \pm u'_j$, etc.). Excluding the bad challenge collisions adds a factor $O(1/|\mathbb{F}|)$ to the knowledge error. □

The full extraction requires three transcripts at each round, giving a tree of 3^k leaves and $O(d)$ prover invocations. In the AGM, each prover invocation is direct (the extractor reads the algebraic representation); in the standard model with forking, each invocation costs $1/p^*$ expected queries to the prover, yielding total cost $O(d \cdot \mathsf{poly}(\lambda)/p^*)$.

3.4 Concrete versus asymptotic security

The literature on knowledge soundness contains numerous tightness gaps. The more important ones include:

- Bulletproofs' standard-model knowledge-soundness reduction loses a factor exponential in the round count $\log d$ (equivalently, quasi-polynomial $d^{O(\log d)}$ in the degree bound d).
- Halo 2's deployed parameters assume the AGM and/or the random oracle model heuristically.
- PLONK's original ZK simulation had a gap, since fixed; see Sefranek, "How (Not) to Simulate PLONK" (SCN 2024).

This reflects the practical reality of deployed SNARKs: tight reductions in idealised models, loose reductions in standard models, and deployment selecting the former.

4 PCD, IVC, and accumulation schemes

We now turn to the most modern part of the foundations: recursive proof composition. This is what makes Halo 2's "no trusted setup with recursion" combination possible, and what underlies any rollup architecture.

4.1 Incrementally-verifiable computation

Definition 4.1 (IVC, informal)**.** An *incrementally-verifiable computation* (IVC; Valiant, TCC 2008) is a scheme for proving the correctness of a computation $z_n = f^{(n)}(z_0) := f(f(\dots f(z_0) \dots))$ such that at each step $z_{i+1} = f(z_i)$, the prover produces a proof π_{i+1} asserting that z_{i+1} is the correct $i+1$-st iterate. The prover at step $i+1$ has access to π_i and produces π_{i+1} in time independent of i (up to logarithmic factors).

The classical approach to IVC is *recursive SNARK composition*: at step $i+1$, prove "$z_{i+1} = f(z_i)$ AND there exists π_i accepting against the verifier circuit for step i." The proof π_{i+1} thereby attests to the full chain of steps, recursively.

For this to work, the SNARK's verifier must be efficiently expressible as a circuit. This is non-trivial: pairing-based SNARKs require an in-circuit pairing check (expensive), while STARK verifiers require in-circuit hash computations (also expensive). The Halo line addresses this differently.

Definition 4.2 (PCD, informal)**.** *Proof-carrying data* (PCD; Chiesa, Tromer; ICS 2010) generalises IVC: instead of a linear chain of computations, the protocol supports an arbitrary DAG of computations, each of which may consume multiple parent proofs and produce one descendant proof.

A rollup chain of N transactions is naturally a PCD instance: each block consumes the previous block's proof plus the new transaction batch, and produces a single proof attesting to the entire history.

4.2 Naive recursion and its failure to scale

The naive approach to IVC proves, at step i, the statement "$z_i = f(z_{i-1})$ AND $\mathsf{Verify}(\mathsf{vk}, z_{i-1}, \pi_{i-1}) = 1$," compiled to a SNARK.

This succeeds in principle but encounters two practical obstacles:

1. For pairing-based SNARKs, the verifier circuit contains a pairing equation, which is costly to express algebraically inside a circuit (it requires extension-field arithmetic).
2. For SNARKs with linear-time verifiers (such as Halo 2's $O(d)$ MSM), the linear-time check dominates the verifier circuit, which negates the per-step efficiency.

The accumulation paradigm addresses (2). For (1), Halo / Halo 2 avoids the issue by using IPA instead of pairings.

4.3 Accumulation schemes

The central insight, due to Bowe–Grigg–Hopwood for Halo and which BCMS20 formalised, is the following.

Definition 4.3 (Accumulation scheme, informal)**.** For a predicate Φ (for example, "this IPA opening is correct"), an *accumulation scheme* consists of:

- An *accumulator*, a compact data structure representing a batch of claims about Φ.
- A *folding* algorithm: given an old accumulator and a new claim about Φ, it produces a new accumulator that morally "contains" both. The folding is fast (sub-linear in the claim size).
- A *decider*, which is expensive (linear-time) but which the scheme invokes only once at the very end to validate the accumulator. If the decider accepts, every claim folded in so far is true.

Theorem 4.4 (BCMS20 accumulation, informal)**.** *For the IPA-PCS, there exists an accumulation scheme in which:*

- *An accumulator is a tuple* $\mathsf{Acc} := (G^{(0)}, u_1, \ldots, u_k) \in \mathbb{G} \times \mathbb{F}^k$*, representing the claim that* $G^{(0)} = \mathsf{Commit}(g(X; u_1, \ldots, u_k))$*.*
- *Folding two accumulators runs in* $O(\log d)$ *time, producing a new accumulator that is valid iff both inputs were.*
- *The decider performs one* $O(d)$*-size MSM to check validity.*
- *Soundness reduces to DL on* $\mathbb{G}$ *in the random oracle model.*

This resolves the IVC problem in the IPA setting: the in-circuit verifier need only check the $O(\log d)$ "succinct" parts of an IPA proof and fold the $O(d)$ "deferred" MSM check into a running accumulator. The decider pays the deferred work once, at the end of the entire chain. Theorem 5.5 restates this concretely once we have the IPA construction in hand.

4.4 The necessity of a cycle of curves

The accumulation scheme's in-circuit folding involves arithmetic on group elements of the underlying curve. For this to be efficient inside the circuit, the group's base field must equal the circuit's native field. Since the circuit's native field is the scalar field of the curve on which the prover commits the SNARK — which in turn is generally different from the curve's own base field — a difficulty arises: an E_p-IPA proof's group elements have coordinates in $\mathbb{F}_p$, so verifying it natively would require a circuit whose native field is $\mathbb{F}_p$. But a SNARK's verifier circuit naturally operates over the curve's *scalar* field $\mathbb{F}_q$ (where $q = |E_p|$): the Fiat–Shamir challenges and folding scalars are $\mathbb{F}_q$ elements. Since a single curve cannot have $\mathbb{F}_p = \mathbb{F}_q$, no one curve E_p can host both the proof's coordinates and its own verifier circuit natively.

The resolution is a *2-cycle* of elliptic curves, where $|E_p| = q$ and $|E_q| = p$. Then:

- A circuit committed on E_q verifies a proof generated over E_p; that circuit's native field is the scalar field of E_q, namely $\mathbb{F}_p$ (since $|E_q| = p$) — which coincides with the coordinate field of E_p, enabling native manipulation of E_p's group elements.
- A circuit over $\mathbb{F}_q$ verifies the next proof, generated over E_q.
- Proofs alternate between the two curves.

This is precisely the Pasta cycle of the *Math Guide*, where the two curves are Pallas (over $\mathbb{F}_{p_{\mathsf{Pallas}}}$, order p_{Vesta}) and Vesta (over $\mathbb{F}_{p_{\mathsf{Vesta}}}$, order p_{Pallas}).

4.5 Connection to the Halo 2 protocol

The Halo 2 protocol (§5.6) ends each proof with a deferred-MSM relation $G^{(0)} \stackrel{?}{=} \mathsf{Commit}(g)$. In a non-recursive deployment (the current Orchard), the verifier pays this MSM as part of the standard verification cost. In a recursive deployment, an accumulator absorbs the same MSM check, defers it across a chain of proofs, and the final decider pays it alone.

The Pasta cycle renders the folding step efficient. In a recursive deployment, a Vesta-IPA proof contains in its circuit a verifier of the previous Pallas-IPA proof. The Vesta circuit's native field is $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (the Vesta scalar field), which coincides with the coordinate field of Pallas points. The in-circuit verifier can therefore manipulate Pallas group elements (cm, rk, $\mathsf{cv}^{\mathsf{net}}$, the accumulator's $G^{(0)}$) using native field arithmetic. The next proof, a Pallas-IPA proof with native field $\mathbb{F}_{p_{\mathsf{Vesta}}}$, verifies that Vesta proof using the symmetric argument. This "cross-curve hop" between Pallas and Vesta is what enables proof-carrying data without expensive non-native field simulation.

5 The Halo 2 proof system

We now assemble the abstractions of the *Crypto Guide*–§4 into the concrete proof system Halo 2: a transparent PLONKish SNARK whose polynomial commitment is the inner-product argument over the Pasta cycle. This section develops Halo 2 to the level of detail required to state *what an Orchard Action proof actually is*, the subject of the *Orchard Guide*.

5.1 PLONKish arithmetisation

Definition 5.1 (PLONKish constraint system)**.** Fix a prime field $\mathbb{F}$ and an integer k defining a multiplicative subgroup $H := \{\omega^0, \ldots, \omega^{n-1}\} \subset \mathbb{F}^*$ of size $n := 2^k$, where ω is a primitive n-th root of unity. A *PLONKish circuit* consists of:

- Columns partitioned into *fixed* (preprocessed), *advice* (prover-witness), and *instance* (public-input).
- A maximum constraint degree d.

- A finite sequence of *custom gates* $g_t(\vec{x}_0, \vec{x}_1, \dots) = 0$ — multivariate polynomials of degree $\le d$ in column values at the current row and at offsets $\omega^r X$ for fixed small r.
- A subset $S \subseteq H \times [\#\text{cols}]$ of *equality-constraint cells* controlled by a permutation σ.
- A set of *lookup arguments*: input tuples $(A_0, \dots, A_{m-1})$ and table tuples $(S_0, \dots, S_{m-1})$, with the assertion that for every row $i \in [0, n)$, $(A_0(\omega^i), \dots, A_{m-1}(\omega^i))$ matches some row of the table.

Vanilla PLONK (Gabizon–Williamson–Ciobotaru, 2019) employs a fixed five-selector universal gate

$$q_M(X)a(X)b(X) + q_L(X)a(X) + q_R(X)b(X) + q_O(X)c(X) + q_C(X) \equiv 0 \pmod{Z_H(X)},$$

where $Z_H(X) := X^n - 1$ is the *vanishing polynomial* of the domain H (it is zero exactly on H, so "$\equiv 0 \bmod Z_H$" means "zero at every row"). "PLONKish" generalises this by permitting designer-chosen gates of higher arity and degree, each gated by a selector $q_i(X)$ vanishing on rows where the gate is inactive. The Orchard Action circuit uses ten advice columns plus a small fixed-column lookup table for Sinsemilla; its custom gates encode the algebraic constraints of the Action statement (the *Orchard Guide*) together with the incomplete-addition arithmetic for in-circuit elliptic-curve operations.

Polynomial representation. The Lagrange basis $\ell_j(X)$ for H, where $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ for $j' \neq j$, interpolates each column into a polynomial of degree $< n$ over H. The prover commits to each polynomial via the IPA-PCS (§5.4). All subsequent arguments are *polynomial identities* on these committed polynomials, verifiable by querying them at random points.

5.2 The permutation (equality) argument

A PLONK-style permutation argument, which Halo 2 generalises to multiple columns, enforces equality constraints between distinct cells. Each cell (i, j) (column i, row j) receives a unique label $\delta_i \cdot \omega^j$ with δ_i chosen so all products are distinct as (i, j) ranges over the trace. The prover writes σ-permuted polynomials $s_i(X)$ with $s_i(\omega^j) = \delta_{i'} \cdot \omega^{j'}$ when $\sigma(i, j) = (i', j')$.

Given Fiat–Shamir challenges $\beta, \gamma \in \mathbb{F}$, the prover commits to a running-product polynomial $Z_{\text{perm}}(X)$ asserting

$$\prod_{i,j} \frac{v_i(\omega^j) + \beta\, \delta_i\, \omega^j + \gamma}{v_i(\omega^j) + \beta\, s_i(\omega^j) + \gamma} = 1.$$

Two values appear in equality-constrained cells iff this product over the trace equals 1. The verifier checks this by opening Z_{perm} at x and ωx, verifying the boundary $Z_{\text{perm}}(\omega^0) = 1$ and the row-by-row update. The per-proof verifier work is $O(m + \log n)$.

5.3 The lookup argument

Halo 2's lookup argument is structurally simpler than plookup (Gabizon–Williamson 2020) and supports arbitrary (possibly non-fixed) tables.

Objective. Establish that every entry of column $A(X)$ on the active domain appears as some entry of column $S(X)$.

Construction. The prover supplies polynomials $A'(X), S'(X)$ where (i) A' is a permutation of A with like values grouped into contiguous runs, and (ii) S' is a permutation of S in which the first row of each run of like values in A' has the matching value in S'. The prover commits to a running-product column $Z_{\text{lookup}}(X)$, then after challenges β, γ proves:

$$\begin{aligned} \ell_0(X)\big(1 - Z_{\text{lookup}}(X)\big) &= 0, \\ Z_{\text{lookup}}(\omega X)(A'(X)+\beta)(S'(X)+\gamma) - Z_{\text{lookup}}(X)(A(X)+\beta)(S(X)+\gamma) &= 0, \\ \ell_0(X)(A'(X) - S'(X)) &= 0, \\ (A'(X) - S'(X))(A'(X) - A'(\omega^{-1}X)) &= 0. \end{aligned}$$

The first three force A' to be a permutation of A and to start each run aligned with S'. The fourth constrains every subsequent row to either start a new run ($A'_i = S'_i$) or continue the previous one ($A'_i = A'_{i-1}$). Inductively, every value of A equals some value of S.

A random-linear combination with a challenge θ reduces multi-column lookups to single-column: $A_{\text{c}}(X) := \sum_j \theta^{m-1-j} A_j(X)$. Variable tables (where S is itself an advice column) and range checks (where S enumerates the allowed range) are special cases.

5.4 The inner-product polynomial commitment

The Halo 2 polynomial commitment, instantiating the abstract PCS of the *Crypto Guide*, is the inner-product argument of Bootle–Cerulli–Chaidos–Groth–Petit (EUROCRYPT 2016) as refined in Bulletproofs (Bünz et al., 2017). We give the construction in detail. Write $d := 2^k$ for the degree bound (equivalently, the number of generators); in the assembled protocol of §5.6 the committed polynomials have degree $< n$, so $d = n$ there.

Construction 5.2 (IPA-based polynomial commitment)**.** Let $\mathbb{G}$ be a cyclic group of prime order p in which DL is hard. Choose $d = 2^k$ and sample, via hash-to-curve, $\vec{G} = (G_1, \dots, G_d)$ and $H \in \mathbb{G}$ with no known non-trivial linear relation among them.

For $p(X) = \sum_{i=0}^{d-1} a_i X^i \in \mathbb{F}_p[X]$ and blinder $r \in \mathbb{F}_p$,

$$\mathsf{Commit}(p; r) := \langle \vec{a}, \vec{G} \rangle + [r]H \in \mathbb{G}.$$

To open p at $x \in \mathbb{F}_p$ with claimed value v, the relation is

$$\mathcal{R}_{\text{IPA}} := \Big\{ \big((P, x, v); (\vec{a}, r)\big) : P = \langle \vec{a}, \vec{G} \rangle + [r]H,\ v = \langle \vec{a}, \vec{b}(x) \rangle,\ \deg p \le d-1 \Big\},$$

with $\vec{b}(x) := (1, x, x^2, \dots, x^{d-1})$.

5.4.1 The recursive halving protocol

The argument runs in $k = \log_2 d$ rounds. Initialise $\vec{a}^{(k)} := \vec{a}$, $\vec{G}^{(k)} := \vec{G}$, $\vec{b}^{(k)} := \vec{b}(x)$, and let $U \in \mathbb{G}$ be a challenge group element (in Fiat–Shamir, derived from the transcript). The prover absorbs $[v]U$ so that the relation becomes

$$P + [v]U = \langle \vec{a}, \vec{G} \rangle + [r]H + [\langle \vec{a}, \vec{b} \rangle]U.$$

In round j (descending from k to 1): split each vector into halves and let

$$\begin{aligned} L_j &:= \langle \vec{a}^{(j)}_{\text{lo}}, \vec{G}^{(j)}_{\text{hi}} \rangle + [\ell_j]H + [\langle \vec{a}^{(j)}_{\text{lo}}, \vec{b}^{(j)}_{\text{hi}} \rangle]U, \\ R_j &:= \langle \vec{a}^{(j)}_{\text{hi}}, \vec{G}^{(j)}_{\text{lo}} \rangle + [r_j]H + [\langle \vec{a}^{(j)}_{\text{hi}}, \vec{b}^{(j)}_{\text{lo}} \rangle]U. \end{aligned}$$

The verifier returns a uniform $u_j \in \mathbb{F}_p^*$. Both parties fold:

$$\vec{a}^{(j-1)} := u_j\,\vec{a}_{\mathrm{lo}}^{(j)} + u_j^{-1}\,\vec{a}_{\mathrm{hi}}^{(j)},$$
$$\vec{G}^{(j-1)} := u_j^{-1}\,\vec{G}_{\mathrm{lo}}^{(j)} + u_j\,\vec{G}_{\mathrm{hi}}^{(j)},$$
$$\vec{b}^{(j-1)} := u_j^{-1}\,\vec{b}_{\mathrm{lo}}^{(j)} + u_j\,\vec{b}_{\mathrm{hi}}^{(j)}.$$

After k rounds each vector has length 1. The prover sends the final scalar $a := \vec{a}^{(0)}$ and an aggregated blinder r^*. The verifier accepts iff

$$P + [v]U + \sum_{j=1}^{k}\big([u_j^2]L_j + [u_j^{-2}]R_j\big) \stackrel{?}{=} [a]\,G^{(0)} + [r^*]H + [a\cdot b^{(0)}]U, \tag{1}$$

where $G^{(0)} = \langle \vec{s},\, \vec{G}\rangle$ and the challenges determine $\vec{s}$ as below.

5.4.2 The structured scalars and the polynomial g

Write the binary expansion $i = \sum_{\ell=0}^{k-1} b_\ell 2^\ell$ for $i \in [0, d)$. By induction on the folding,

$$G^{(0)} = \langle \vec{s},\, \vec{G}\rangle, \qquad b^{(0)} = \langle \vec{s},\, \vec{b}(x)\rangle = g(x; u_1, \ldots, u_k),$$

where

$$s_i := \prod_{\ell=0}^{k-1} u_{\ell+1}^{(-1)^{1-b_\ell}}, \qquad g(X; u_1, \ldots, u_k) := \prod_{\ell=1}^{k}\big(u_\ell^{-1} + u_\ell X^{2^{\ell-1}}\big).$$

The essential asymmetry is the following: $g(X)$ has degree $d-1$ and the verifier can evaluate it at x in $O(\log d)$ field operations; but computing $G^{(0)} = \mathsf{Commit}(g)$ requires an $O(d)$-size multi-scalar multiplication (MSM). This asymmetry is the seed of the accumulation scheme (§5.5).

5.4.3 Worked example: an IPA opening at $d = 4$

Take $d = 4$, so $k = 2$ rounds. Let $p(X) = a_0 + a_1X + a_2X^2 + a_3X^3$ with $\vec{a} = (a_0, a_1, a_2, a_3)$. The verifier holds $P = \langle \vec{a},\, \vec{G}\rangle + [r]H$ (we suppress r to declutter; the H-part propagates identically) and requires $v = p(x)$ at some $x \in \mathbb{F}_p$.

Set $\vec{b}^{(2)} := (1, x, x^2, x^3)$ and $\vec{G}^{(2)} := \vec{G}$. After the prover absorbs $[v]U$, the relation is $P + [v]U = \langle \vec{a},\, \vec{G}\rangle + \langle \vec{a},\, \vec{b}^{(2)}\rangle\, U$.

Round 2. Split into halves of size 2. The prover sends

$$L_2 = a_0G_3 + a_1G_4 + (a_0x^2 + a_1x^3)U,$$
$$R_2 = a_2G_1 + a_3G_2 + (a_2 + a_3x)U.$$

The verifier picks $u_2 \in \mathbb{F}_p^*$ and both parties fold to length-2 vectors as above.

Round 1. Split the length-2 vectors. The prover sends L_1, R_1 analogously; the verifier picks u_1. After folding, all three vectors have length 1.

Final check. With $i = b_0 + 2b_1$ ranging over $\{0, 1, 2, 3\}$:

$$s_0 = u_1^{-1}u_2^{-1}, \qquad s_1 = u_1u_2^{-1}, \qquad s_2 = u_1^{-1}u_2, \qquad s_3 = u_1u_2.$$

The verifier evaluates $g(x; u_1, u_2) = (u_1^{-1} + u_1x)(u_2^{-1} + u_2x^2)$ in two multiplications, and computes $G^{(0)} = \sum_{i=0}^{3}[s_i]G_{i+1}$ as a length-4 MSM. The check (1) becomes

$$P + [v]U + [u_1^2]L_1 + [u_1^{-2}]R_1 + [u_2^2]L_2 + [u_2^{-2}]R_2 \stackrel{?}{=} [a]G^{(0)} + [a\cdot g(x; u_1, u_2)]\,U.$$

The folding rule for $\vec{G}$ ensures that $G^{(0)}$ is a specific linear combination of the original $\vec{G}$ with coefficients $\vec{s}$; that the s_i match $\prod_\ell (u_\ell^{-1}$ or $u_\ell)$ according to the binary expansion of i is exactly the closed-form identity. The verifier can evaluate g in $\log d$ field operations but recomputing $G^{(0)}$ takes $O(d)$ MSM — the asymmetry the accumulator defers.

5.4.4 Knowledge soundness via rewinding

Theorem 5.3 (Knowledge soundness of IPA)**.** *Under the DL assumption on $\mathbb{G}$, and modelling Fiat–Shamir as a random oracle, Construction 5.2 is knowledge-sound for $\mathcal{R}_{\text{IPA}}$ with knowledge error $O(d/|\mathbb{F}_p|)$.*

Sketch of the tree extractor. The extractor rewinds the prover $\mathcal{P}^*$ to obtain, at each round, two transcripts that agree on (L_j, R_j) but differ in u_j. The two corresponding verifier equations form a 2×2 linear system in the round-j halved witness whose Vandermonde determinant is $(u_j^2 - u_j'^2)/(u_j u_j')$, non-zero whenever $u_j \neq \pm u_j'$. This recovers the witness halves $\vec{a}_{\text{lo}}^{(j)}, \vec{a}_{\text{hi}}^{(j)}$.

Extracting the full witness $\vec{a} \in \mathbb{F}_p^d$ requires two successful transcripts at the deepest level, then for *each pair*, two transcripts at the next-shallower level, and so on. The tree has $2^k = d$ leaves, yielding polynomial expected extraction time when $d = \mathsf{poly}(\lambda)$. The knowledge error of $O(d/|\mathbb{F}_p|)$ accounts for the probability that any $2d$ challenges in the tree happen to collide, which a union bound renders negligible.

This is the recursive generalisation of 2-special soundness for Schnorr (the *Crypto Guide*): a single transcript reveals nothing, but two transcripts on the same first message permit inversion of a 2×2 system; in IPA the inversion is structured but the principle is identical. □

5.5 Accumulation and the Halo trick

The verifier's check (1) runs in $O(\log d)$ scalar operations and $\log d$ point additions, *except* for the single evaluation $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ — an MSM of length d. The accumulation idea (Bowe–Grigg–Hopwood 2019; formalised by BCMS20) is to *defer* this MSM and amortise it across many proofs.

Definition 5.4 (Atomic accumulator for IPA)**.** An IPA *accumulator instance* is a tuple $\mathsf{Acc} := (G^{(0)}, u_1, \ldots, u_k) \in \mathbb{G} \times \mathbb{F}_p^k$. It is *valid* iff $G^{(0)} = \mathsf{Commit}(g(X; u_1, \ldots, u_k))$.

Theorem 5.5 (Accumulation of IPA, BCMS20)**.** *There is an accumulation scheme* $(\mathsf{AccVerifier}, \mathsf{AccDecider})$ *for IPA with:*

- $\mathsf{AccVerifier}$*, on input a new IPA instance and an old accumulator, runs in $O(\log d)$ and outputs a new accumulator* Acc'*.*
- $\mathsf{AccDecider}$*, run* once *at the end of a chain, performs an MSM of length d to check $G^{(0)} = \mathsf{Commit}(g)$.*
- *Soundness reduces to DL on $\mathbb{G}$ in the random-oracle model.*

Operational interpretation. A Halo 2 verifier circuit, expressed over the scalar field of one Pasta curve and verifying a proof produced over the *other* curve, can:

1. Run the $O(\log d)$ "succinct" checks of the IPA verifier natively.
2. Witness the next-step claimed $G^{(0)\prime}$ and the next challenges $u_1', \ldots, u_k'$ as public inputs to the next proof, deferring the MSM check.
3. Fold the deferred MSM checks: by linear combination with a random challenge ρ, reduce two MSM checks to one, $\mathsf{Acc}'' := \mathsf{Acc} + \rho \cdot \mathsf{Acc}'$.

A single final MSM at the end of the chain validates the entire history. The Pasta cycle ensures the in-circuit scalar arithmetic on each step is native, as we develop in §4.

The mathematical core is the recognition that $G^{(0)} = \mathsf{Commit}(g(X; u_1, \ldots, u_k))$ — the linear-time check is *itself a polynomial commitment opening of a structured polynomial* and hence amenable to recursion within an IPA.

5.6 The complete Halo 2 protocol

We now assemble the pieces into the seven-phase protocol implemented in the halo2 codebase. Throughout, $\mathbb{F}$ denotes the constraint field (Orchard: $\mathbb{F}_{p_{\mathsf{Pallas}}}$), $\mathbb{G}$ the IPA group (Orchard: $\mathsf{Vesta}(\mathbb{F}_{p_{\mathsf{Vesta}}})$), and H the multiplicative subgroup of $\mathbb{F}$ of order n.

Setup (preprocessed, one-time). The circuit designer specifies columns, custom gates, lookup tables, and the equality permutation σ. Both parties derive commitments to the fixed-column polynomials $f_i(X)$, selector polynomials $q_i(X)$, permutation polynomials $s_i(X)$, and lookup-table polynomials $S_j(X)$, together with the Lagrange basis on H, the vanishing polynomial $Z_H(X) := X^n - 1$, and the IPA generators $\vec{G} \in \mathbb{G}^n$ and $H \in \mathbb{G}$.

Phase 1 — advice commitments. The prover constructs the advice column values $\vec{w_i} \in \mathbb{F}^n$ satisfying all gates and equality constraints; interpolates each $\vec{w_i}$ to a polynomial $w_i(X)$ of degree $< n$ on H; adds a random low-degree blinding polynomial for ZK; commits via IPA-PCS; sends the commitments.

Phase 2 — lookup challenges θ, β, γ. For each lookup, the verifier (via Fiat–Shamir) returns challenges: θ to compress multi-column lookups, β, γ to instantiate the running-product. The prover commits to $A', S', Z_{\text{lookup}}$.

Phase 3 — permutation argument. The prover constructs $Z_{\text{perm}}(X)$ as in §5.2 and commits to it.

Phase 4 — quotient challenge y. The verifier returns a fresh $y \in \mathbb{F}$. The prover assembles the *gate equation*: a single polynomial $C(X)$ that is a y-weighted linear combination of (i) each custom gate, (ii) permutation constraints, (iii) lookup constraints. By construction $C(X)$ vanishes on H when all gates pass. The prover computes the quotient $h(X) := C(X)/Z_H(X) = C(X)/(X^n - 1)$, splits h into chunks of degree $< n$, and commits to each chunk.

Phase 5 — evaluation challenge x. The verifier returns $x \in \mathbb{F}$. The prover evaluates every committed polynomial at x, and at ωx for adjacent-row references, and sends these claimed evaluations as field elements. The verifier checks the gate equation $C(x) = h(x)Z_H(x)$ as a numerical identity in the sent evaluations. This check is meaningful only once the claimed evaluations bind to the commitments, which is the role of phases 6–7; a prover who sent inconsistent evaluations would pass this field identity but fail the opening argument.

Phase 6 — batched polynomial opening. To verify the evaluations are consistent with the commitments, the prover runs Halo 2's multipoint-opening argument: it groups the polynomials queried at the same point set and combines them via a challenge x_1, then builds a single polynomial $f^*(X)$ representing all openings such that opening f^* at one final point x_3 implies all original openings.

Phase 7 — IPA opening. Prover and verifier execute the recursive halving IPA (§5.4) on f^*, yielding $\log_2 n$ pairs (L_j, R_j), a final scalar a, and a final blinder r^*. The verifier accepts iff (1) holds.

Accumulation (recursion). The next proof's verifier circuit *defers* the MSM check $G^{(0)} = \mathsf{Commit}(g(X; u_1, \ldots, u_k))$ at the end of phase 7 into an accumulator and folds it with prior accumulators (§5.5). A single decider at the very end validates the entire chain.

Costs. Per proof, ignoring the deferred MSM: $O(\log n)$ pairs checked in phase 7; one gate-equation field check; a constant number of polynomial evaluations consumed. Without accumulation, the verifier additionally performs one $O(n)$-size MSM; with accumulation the chain pays for this MSM once at the end. FFTs and one $O(n)$ MSM dominate the prover's work (typical Orchard Action: $n = 2^{11}$, a few seconds on commodity hardware).

5.7 Instance columns and binding the public statement

Definition 5.1 lists *instance* columns alongside fixed and advice columns, but the arguments of §5.2–§5.6 never single them out. We single them out now, because what an Orchard Action proof *asserts* is precisely a statement about its instance values — the anchor, nullifier nf, net value commitment $\mathsf{cv}^{\mathsf{net}}$, randomised key rk, and the $\mathsf{enableSpends}/\mathsf{enableOutputs}$ flags (the *Orchard Guide*).

The instance polynomial. An instance column is a public-input column: its n entries $\vec{\pi} = (\pi_0, \ldots, \pi_{n-1}) \in \mathbb{F}^n$ are not part of the witness but both parties agree on them before verification. Like every other column it is interpolated over H in the Lagrange basis,

$$\pi(X) := \sum_{j=0}^{n-1} \pi_j \, \ell_j(X) \in \mathbb{F}[X], \qquad \deg \pi < n,$$

where $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ otherwise (§5.1). The prover writes the public values into a handful of designated rows; all remaining π_j are 0. A custom gate of the form $q(X)\big(a(X) - \pi(X)\big) = 0$, with q a selector active exactly on those rows, copies an advice cell to a public value and thereby asserts "advice cell $= \pi_j$." In Orchard, the gate constrains the cells carrying the in-circuit nf, $\mathsf{cv}^{\mathsf{net}}$, rk and the flags against the instance column this way, so a satisfying assignment exists *only* for the public values the verifier supplied.

Where it enters the protocol. Unlike advice, the prover never commits to $\pi(X)$ and never includes it among the IPA openings of phases 6–7: the verifier already holds $\vec{\pi}$, so committing would be redundant and would disclose nothing. The instance polynomial enters only the *gate equation*. When the verifier checks $C(x) = h(x)\, Z_H(x)$ at the evaluation challenge x (Phase 5, §5.6), the assembled $C(x)$ contains the term $q(x)\big(a(x) - \pi(x)\big)$ from the gate above. The advice evaluation $a(x)$ arrives in the transcript and the opening argument binds it to its commitment; the selector evaluation $q(x)$ is a fixed-column value both parties recompute. The verifier supplies the remaining evaluation $\pi(x)$ itself, computing $\pi(x) = \sum_j \pi_j \, \ell_j(x)$ directly from the known public values — an $O(\log n)$ evaluation using the barycentric form of the Lagrange basis on H, never read from the proof.

Consequence. A prover who substitutes different public values $\vec{\pi}\,'$ must still satisfy $C(x) = h(x)\, Z_H(x)$ for the $\pi(x)$ the *verifier* computes from the genuine $\vec{\pi}$. Knowledge soundness (§3) then extracts a witness for the relation instantiated at exactly those public values; no witness exists for the wrong ones except with negligible probability. This binds the proof to its public statement: an Action proof that a node verifies against an anchor it accepts, a nullifier it will mark spent, and a value commitment it will sum into the net balance check (the *Orchard Guide*) can only verify if the prover knew a witness consistent with *those* values.

5.8 Zero knowledge: hiding the witness in Halo 2

Knowledge soundness (§3) establishes that a convincing prover *knows* a witness; it remains to explain why the proof *reveals* no more than that. Halo 2 is zero-knowledge through two devices — hiding commitments and blinded polynomials — assembled into a simulator.

Hiding commitments. The polynomial commitment of §5.4 is *perfectly hiding*. The prover commits a polynomial p as

$$\mathrm{Commit}(p; r) \;=\; \langle \vec{p},\, \vec{G} \rangle + [r]\, H, \qquad r \xleftarrow{\$} \mathbb{F}_p,$$

with $\vec{p}$ the coefficient vector of p and H an independent generator of no known discrete-log relation to $\vec{G}$. The $[r]H$ term makes the commitment a *uniform* group element for every p, so a commitment by itself leaks nothing about what the prover committed (the perfect hiding of a Pedersen commitment, *Crypto Guide*). Every commitment the verifier sees — the advice columns, the permutation and lookup products, the quotient chunks — has this form.

Blinding the witness polynomials. The prover eventually *opens* a commitment, and an opening reveals evaluations. The prover interpolates the advice columns over the domain H to polynomials of degree $< n$; to prevent those evaluations from leaking the witness, the prover does not use all n rows for circuit data. It reserves a small fixed number of rows at the bottom of the table — the *blinding rows*, never covered by an active gate — and fills them with uniform random field elements, at least one more than the number of times the verifier will open any single column. Its n evaluations pin down a degree-$(< n)$ polynomial, so with enough independent random values planted on the blinding rows the handful the verifier actually queries are jointly uniform and independent of the circuit data. The prover blinds the permutation and lookup running products and the quotient pieces the same way.

Why the openings leak nothing. In each phase the verifier sees only (i) hiding commitments — uniform group elements — and (ii) a bounded number of evaluations of the committed polynomials at the Fiat–Shamir challenge points. The blinding rows make each such evaluation uniformly random subject only to the *public* relations the verifier itself checks; no information about the private witness passes through. The final inner-product opening (§5.4) likewise runs with a blinder, so it too hides its input.

The simulator. Formally, zero knowledge is the existence of a *simulator* that, given only the public input and *not* the witness, outputs a transcript indistinguishable from a real one. Halo 2's simulator runs the protocol "backwards," exactly as Schnorr's does (*Crypto Guide*): it chooses the evaluations and the final opening so as to satisfy the verifier's equations, then chooses hiding commitments consistent with them — possible precisely because the commitments are perfectly hiding and it may choose the blinders freely — and, non-interactively, *programs* the Fiat–Shamir oracle so the challenges come out as required. Since the commitments are perfectly hiding and a blinder protects every opening, the simulated transcript is distributed as a real one. Interactively this is honest-verifier zero knowledge; compiled through Fiat–Shamir in the random-oracle model it is a NIZK that reveals nothing beyond the truth of the statement.

The three properties together. For the relation fixed by a PLONKish circuit, Halo 2 thus delivers all three guarantees a SNARK should: *completeness* (an honest prover holding a satisfying witness always convinces the verifier), *knowledge soundness* (§3: from any convincing prover an extractor pulls a satisfying witness, so an accepted proof certifies one is *known*), and *zero knowledge* (a witness-free simulator reproduces the proof, so it leaks nothing else). It is exactly this combination — "the prover knows a satisfying witness, and the proof discloses nothing more"

— on which the Orchard Action proof rests: knowledge soundness yields balance and nullifier integrity, zero knowledge yields privacy (*Orchard Guide*).